Information Processing and Retrieval Project

CATARINA GONÇALVES* and ISABEL COSTA*, ist1103989 and ist1106868

1 Supervised IR using reference summaries

1.1 Methodology & Design Decisions

The supervised component explores two strategies to leverage reference extracts as feedback: fine-tuning a pretrained decoder LLM and reformulating summarization as a learning-to-rank classification task. The dataset was split into 80% training (1780 documents) and 20% testing (445 documents) using stratified sampling to preserve the original category distribution across both sets.

`fine-tuning`$(M, D_{train}, R_{train}, \textbf{args})$

A `t5-small` model was selected for fine-tuning due to its lightweight sequence-to-sequence architecture (which relies on an autoregressive decoder), and its extensive pre-training on summarization corpora, making it a natural fit for the task without prohibitive computational requirements [Raffel et al. 2020]. Input documents were truncated to 512 tokens and target summaries to 150 tokens. An internal 90/10 train-validation split was applied to monitor generalization, with the validation set capped at 100 samples to prevent out-of-memory errors during evaluation.

The fine-tuning pipeline was adapted from Lab 4 [PRI Lab 4 2026] and implemented using the Hugging Face `Seq2SeqTrainer` [Hugging Face 2024; Wolf et al. 2020]. Performance was tracked using ROUGE scores computed at the end of each epoch, through `compute_metrics`, ensuring padding tokens (-100) were replaced before decoding. The values for the learning rate (5×10^{-5}), batch size (4), and epochs (5), were selected based on the ROUGE-L performance across configurations (see Section 1.2). The best checkpoint was retained via `load_best_model_at_end=True`. Finally, summaries were generated for both the baseline `t5-small` and the fine-tuned model on the test set using greedy decoding with `max_new_tokens=150`, and evaluated against reference summaries using ROUGE-1, ROUGE-2, and ROUGE-L.

`feature_extraction`$(s, d, I, \textbf{args})$

Each sentence s from a document d is represented as a 4-dimensional feature vector:

- Feature 1 (Sentence length): Measured as the raw word count.
- Feature 2 (Relative position): Calculated as the normalized index of s within d, capturing the journalistic tendency to place key information early.
- Feature 3 (TF–IDF score): Obtained by summing the TF–IDF weights of the sentence tokens from the collection-level lexical index and normalizing by sentence length.
- Feature 4 (Semantic similarity): Computed as the cosine similarity between the `BAAI/bge-small-en-v1.5` sentence embeddings of s and d.

Features 1 and 2 capture structural properties of the sentence, while features 3 and 4 capture document information. The impact of excluding structural features is assessed in Section 1.2 via an ablation study restricted to features 3 and 4.

`training`$(D_{train}, R_{train}, \textbf{args})$

Label generation follows a soft ROUGE-2 matching strategy: for each document, candidate sentences are scored against the reference summary using ROUGE-2 F-measure, and the top-k sentences are labeled as positive, where k matches the number of sentences in the reference summary. This ensures the classifier learns to select a realistically-sized summary rather than performing unconstrained binary classification. Two classifiers were selected, Random Forest (`n_estimators=100`, `class_weight='balanced'`) and Gaussian Naïve Bayes. These classifiers were chosen for their simplicity and the lack of need for hyperparameter tuning. The `class_weight='balanced'` setting in the Random Forest adjusts class weights inversely proportional to their frequencies, preventing the model from defaulting to the majority, non-summary, class. Cross-validation was not employed due to the high computational cost of feature extraction. While this disables hyperparameter tuning and limits the use of more expressive architectures (such as gradient boosting or neural networks), these represent natural directions for future work. Finally, to avoid redundant computation during experimental runs, the pre-computed feature matrix `X_train` is passed directly via `args`.

`summarization`$(d, M, p, l, \textbf{args})$

The `summarization` function generates an extractive summary of the document d using a trained classifier M. Sentences are first tokenized using `nltk.sent_tokenize`, and the feature extraction function is applied to each sentence to produce the test feature matrix `X_test`. The classifier scores each sentence via `predict_proba`, and sentences are ranked by their probability of belonging to a summary. The top-p sentences are selected, where p is a category-specific limit derived from the Exploratory Data Analysis (EDA) made in Project I. Chronological ordering is then restored by sorting selected sentences by their original position in the document, ensuring the produced summary remains coherent and readable. Only the sentence limit p is enforced. The character limit l was not applied, as p alone was deemed sufficient to control summary length given the category-level calibration.

`supervised_evaluation`$(D_{test}, R_{test}, M, \textbf{args})$

The classifier is evaluated using confusion-based metrics (precision, recall, F1-score, and AUC) computed at the sentence level. Ground-truth labels are generated using the same soft ROUGE-2 matching strategy done for training: for each document, sentences are scored against the reference summary and the top-k are labeled as positive, where k matches the number of sentences in the reference. This consistency between training and evaluation labels ensures that the

*Both authors contributed equally to this research.

classifier is assessed on the same criteria that it was trained to optimize. Sentence-level predictions are obtained via `predict_proba`, with the AUC computed from the predicted probabilities to capture the ability to rank beyond hard classification decisions.

`evaluation(S_test, R_test, args)`

The `evaluation` function from Project I was extended in two ways. First, a structural correction was applied: the function previously received ranked pairs of sentence IDs and scores, instead of the final summary strings directly. The input interface was therefore updated to accept text, with precision and recall computed via exact sentence matching after tokenization. Second, the system was expanded to include ROUGE-1, ROUGE-2, and ROUGE-L scores, as required for Project II.

1.2 Questions

1. Does fine tuning help aligning summaries with reference expectations? According to ROUGE, how the fine-tuning setup (e.g. number of epochs) impacts performance?

To determine whether fine-tuning helped align the generated summaries with reference expectations, we compared ROUGE scores from our best fine-tuned model against the pre-trained `T5-small` baseline.

Model Setup	ROUGE-1	ROUGE-2	ROUGE-L
Base `T5-small`	0.3205	0.2110	0.2339
Fine-tuned `T5-small`	**0.5197**	**0.3904**	**0.3857**

Table 1. Baseline vs. fine-tuned ROUGE scores on the test set for `T5-small`.

As shown in Table 1, fine-tuning consistently improves performance across all ROUGE metrics, with ROUGE-1 increasing by approximately 20 percentage points. This confirms that domain-specific adaptation on the BBC News corpus meaningfully aligns the model's outputs with the reference summaries. The base `T5-small`, while pre-trained on large summarization corpora, generates summaries that do not match the style and vocabulary of BBC reference extracts, resulting in lower n-gram overlap. Fine-tuning bridges this gap by exposing the model to the target domain's linguistic patterns.

To understand how the fine-tuning setup impacts performance, we fine-tuned three variants at different learning rates and tracked their ROUGE-L scores over 5 epochs.

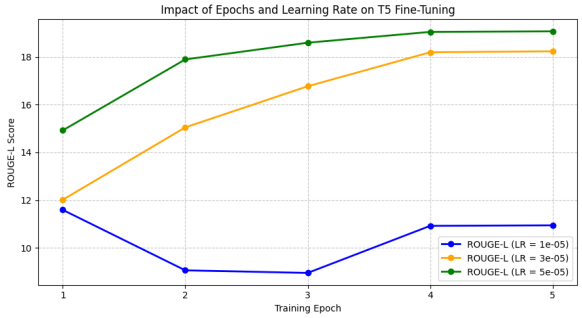


Fig. 1. ROUGE-L score across 5 epochs for three learning rates.

As shown in Figure 1, both the number of epochs and the learning rate significantly impact performance. The learning rate of 5×10^{-5} converged quickly and finished with the highest ROUGE-L score by epoch 5. The mid-range rate of 3×10^{-5} also converged steadily, , reaching a comparable plateau by epoch 4, though slightly below 5×10^{-5}. In contrast, 1×10^{-5} failed to converge, collapsing after the first epoch. This confirms that epoch count matters, but only once a suitable learning rate is established. Additional epochs with an inadequate rate compound errors rather than improve alignment.

One practical constraint worth noting: to avoid out-of-memory errors, the validation set was truncated during evaluation and the batch size was fixed at 4. This kept the fine-tuning process stable, but it does limit what we can conclude. The batch size influences the gradient noise and convergence behavior, and this was never varied. Future work exploring that variance could give a more complete picture of how the training configuration affects final performance.

2. Does the incorporation of feedback from reference extracts significantly impact the performance of the classification-based IR system? Hypothesize why is that so.

The `supervised_evaluation` function assesses the classifiers at the sentence level. Ground-truth labels are generated by ROUGE-2 soft matching, labeling exactly k sentences as positive per document, where k matches the number of sentences in the reference summary. The classifier then predicts through `.predict()`, which applies a fixed 0.5 probability threshold with no awareness of k. This mismatch between how labels are defined and how predictions are made is the primary driver of the results in Table 2 and Table 3: the Random Forest is highly conservative under this threshold, while Naïve Bayes predicts more liberally, achieving a more balanced precision-recall trade-off.

	Random Forest		Naïve Bayes	
	Predicted In	**Predicted Out**	**Predicted In**	**Predicted Out**
Actual In	246	709	385	570
Actual Out	184	7499	586	7097

Table 2. Confusion matrices for sentence inclusion. Rows = Actual, Columns = Predicted.

Table 3 reports the sentence-level metrics. The AUC scores for both models indicate meaningful ranking ability beyond random selection, which is not captured by precision and recall alone under the fixed threshold.

Metric	Random Forest	Naïve Bayes
Precision	0.5721	0.3965
Recall	0.2576	0.4031
F_1-Score	0.3552	0.3998
AUC	0.8253	0.8188

Table 3. Sentence-level supervised evaluation results.

However, `supervised_evaluation` is not the appropriate function for assessing if feedback impacts the performance, since the

unsupervised baselines from Project I operate on a fixed number of sentences, p, and cannot be assessed at the sentence classification level. The `summarization_supervised` function addresses this by bypassing the binary threshold, as sentences are ranked with `predict_proba()` and the top-p are selected per category. This puts all models on equal footing in terms of evaluation. Table 4 compares Project II supervised classifiers (Random Forest and Naïve Bayes) against two unsupervised baselines from Project I (a lexical model (BM25) and the best-performing semantic model, `MiniLM`), evaluated on the exact same holdout test set (D_{test}).

Metric	BM25	MiniLM	Random Forest	Naïve Bayes
Macro Precision	0.3852	**0.5929**	0.4721	0.4356
Macro Recall	0.3121	**0.4865**	0.3893	0.3570
Macro F_β	0.3426	**0.5319**	0.4243	0.3901
Avg. ROUGE-1	0.5674	**0.6245**	0.6045	0.6142
Avg. ROUGE-2	0.4407	**0.5362**	0.4848	0.4948
Avg. ROUGE-L	0.3994	**0.4294**	0.4159	0.4212

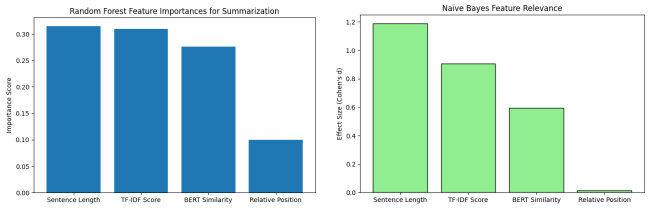
Table 4. Performance comparison between unsupervised (Project I) and supervised (Project II) extractive summarization systems.

As shown in Table 4, reference feedback does improve upon the lexical baseline across all metrics, suggesting the training signal carries genuine value. However, `MiniLM` remains the strongest model overall, outperforming both classifiers on every metric. The impact of feedback cannot therefore be considered significant.

We hypothesize that this is caused by three compounding factors. First, the training data suffers from severe class imbalance (evident in Table 2),as only a small fraction of sentences per document appear in the reference summary, biasing the classifiers toward predicting negative. Second, Random Forest and Naïve Bayes are shallow models lacking the representational capacity to capture the contextual semantics that make `MiniLM` effective. Third, the four features used (sentence length, relative position, TF-IDF score, and BERT-to-document similarity) encode general structural and topical relevance, but not the fine-grained semantic alignment with the reference extract that the labels implicitly require. The classifiers are therefore learning from labels that demand a level of semantic discrimination the feature space cannot express.

3. Which features appear to be more relevant to the target summarization task? Are the learned classifiers able to generalize from one category to another? Justify

Feature relevance was assessed in each model. For Random Forest, we analyzed the Gini impurity-based feature importance. For Naïve Bayes, since it does not expose a direct importance measure, Cohen's d effect size was utilized to measure the standardized mean difference between the summary and non-summary sentence distributions.



1 Random Forest feature importance. 2 Naïve Bayes feature importance.

Fig. 2. Feature importance comparison across models.

As shown in Figure 2, both models agree on the overall feature ranking. Sentence length is considered the most important feature, suggesting that longer sentences carry more information and are more likely to be selected for reference summaries. The TF-IDF score is also considered highly important, capturing topical relevance at the lexical level. BERT similarity, although less important than TF-IDF, adds a valuable semantic dimension by measuring a sentence's alignment with the overall document context. Surprisingly, the low contribution of relative position suggests that, for these specific reference summaries, key sentences are drawn from various locations throughout the text. This deviates from the traditional inverted pyramid structure typically expected in journalistic news.

To assess the contribution of structural features, an ablation study was conducted by retraining both classifiers using only TF-IDF score and BERT similarity, removing sentence length and relative position. Table 5 reports the results (using `evaluation`) against the full-feature models from Table 4.

Metric	RF	RF (Ablation)	NB	NB (Ablation)
Macro Precision	0.4721	0.4775	0.4356	0.4616
Macro Recall	0.3893	0.3941	0.3570	0.3775
Macro F_β	0.4243	0.4293	0.3901	0.4141
Avg. ROUGE-1	0.6045	0.5966	0.6142	0.6082
Avg. ROUGE-2	0.4848	0.4791	0.4948	0.5003
Avg. ROUGE-L	0.4159	0.4084	0.4212	0.4167

Table 5. Full feature set vs. ablation (TF-IDF + BERT only) on the test set.

The differences in Table 5 are minimal across both models, confirming that sentence length and relative position carry limited discriminative value beyond what TF-IDF and BERT similarity already provide. Removing them does not meaningfully degrade performance, and in some metrics the ablation models perform marginally better, suggesting the structural features introduce mild noise rather than useful signal. This aligns with the feature importance analysis: while sentence length ranks highly by Gini impurity, its predictive contribution at the summary level is modest once semantic and lexical relevance are already accounted for.

To determine whether the learned classifiers generalize across categories, two leave-one-category-out experiments were executed. Models were trained on all categories except one, then tested exclusively on the held-out category. This was repeated for *Sport* (shorter,

lighter articles) and *Politics* (longer, denser articles) to assess generalization across contrasting document profiles.

Metric	Sport (held-out)		Politics (held-out)	
	RF	NB	RF	NB
Precision	0.5216	0.5017	0.4108	0.3781
Recall	0.0878	0.2112	0.2235	0.4989
F_1-Score	0.1503	0.2972	0.2895	0.4302
AUC	0.7554	0.7879	0.8335	0.8672

Table 6. Sentence-level supervised evaluation for leave-one-category-out experiments.

The AUC scores in Table 6 indicate that both models retain a meaningful ranking ability when tested on unseen categories, particularly for *Politics* where AUC reaches 0.8672 for Naïve Bayes. However, precision and recall diverge considerably between the two held-out categories. For *Sport*, the Random Forest achieves high precision but very low recall, predicting very few positive sentences under the 0.5 threshold. This is consistent with shorter articles having fewer candidate sentences, making the classifier overly conservative. For *Politics*, Naïve Bayes achieves a notably more balanced trade-off, suggesting it generalizes better to longer, denser documents.

Metric	Sport (held-out)		Politics (held-out)	
	RF	NB	RF	NB
Macro Precision	0.4599	0.4496	0.4754	0.4714
Macro Recall	0.4341	0.4258	0.4487	0.4483
Macro F_β	0.4192	0.4106	0.4558	0.4521
Avg. ROUGE-1	0.5832	0.6056	0.6341	0.6576
Avg. ROUGE-2	0.4766	0.5059	0.5083	0.5402
Avg. ROUGE-L	0.4168	0.4303	0.4043	0.4171

Table 7. Summary-level evaluation for leave-one-category-out experiments.

At the summary level, Table 7 shows that both models generalize reasonably well. Comparing against the full-training results in Table 4, the drop in performance is small, particularly for *Politics* where ROUGE scores are slightly higher. This is likely because political articles share vocabulary and structure with other dense categories such as *Tech* and *Business*, making cross-category transfer more natural. *Sport* shows a slightly lower score, consistent with its structure, shorter, more domain-specific vocabulary, that differ from the underlying structure that dominated the remaining categories.

Overall, the classifiers demonstrate adequate cross-category generalization, which can be partially attributed to the design of `feature_extraction`. Since all features are computed at the sentence-level relative to the enclosing document (rather than against any category-level statistics) the classifier learns a general notion of sentence summary-worthiness that is not tied to domain-specific vocabulary or structure. This makes the learned mapping transferable across categories.

2 Unsupervised IR

2.1 Methodology & Design Decisions

`sentence_clustering(d,I,args)`

The `sentence_clustering` function operates in two primary steps. First, the document is tokenized into discrete sentences and converted into mathematical embeddings. Second, the sentences are grouped using a deterministic hierarchical algorithm that takes into account a specific distance metric and linkage method. The final clustering solution is returned as a dictionary containing the cluster assignments, the extracted sentences, the mathematical embeddings, and the total number of clusters found.

The function accepts a document, an optional pre-computed index (`I`) to bypass computationally heavy encoding, and clustering arguments (`args`). Within `args`, a `SentenceTransformer` encoder, selected for its efficiency in generating dense and high-quality vector representations of text, efficiently generates dense vector representations. If a target cluster count is omitted, the function dynamically evaluates between 3 and 7 cluster-bounds, dictated by average document length.

The optimal cluster count is then selected by maximizing Silhouette scores, ensuring strong intra-cluster cohesion and inter-cluster separation. Finally, the function returns a comprehensive dictionary containing the cluster assignments, extracted sentences, and embeddings, alongside visual diagnostics: a structural dendogram and a Silhouette score line graph, which are analyzed in Section A.1.1.

If the optional index is omitted, the function processes the raw text from scratch. We utilize the Natural Language Toolkit (NLTK) to tokenize the document into discrete sentences. NLTK was chosen because it robustly handles natural language edge cases, such as abbreviations and complex punctuation, ensuring the encoder receives properly formed sentences.

For the clustering phase, we utilized Hierarchical Agglomerative Clustering (HAC). HAC is deterministic, guaranteeing reproducible, bottom-up sentence clusters. To measure similarity, we applied cosine distance, which calculates the angle between vectors rather than their magnitude, accurately capturing semantic meaning regardless of sentence length. Finally, we paired this with average linkage, which evaluates the mean distance across all sentences in merging clusters. This prevents single-pair bias and ensures the resulting groups are truly cohesive and topic-centered.

The final output is the clustering solution (C), a dictionary containing cluster assignments (mapping IDs to sentence indices), raw sentences, vector embeddings, and the total cluster count. Returning all components is essential for the `summarization` phase. The summarizer requires the numerical embeddings to calculate cluster centroids and mathematically identify the most representative sentence index. It then maps this winning index back to the raw sentences to extract the human-readable text for the final summary.

`summarization_clustering(d,C,I,args)`

The `summarization_clustering` function is structured into two primary operations. First, the algorithm identifies a "prototype" sentence for every cluster. Second, these representative sentences are reorganized into their original document order.

To select the most representative sentence per cluster, the function calculates the centroid by averaging the numerical vectors of all sentences within the group. Then it measures cosine distance between this centroid and each individual sentence. Consistent with Section 2.1.1, cosine distance is utilized because it prioritizes semantic orientation over vector magnitude. The sentence closest to the centroid is extracted as the prototype, since it mathematically captures the cluster's core meaning.

To maintain a logical narrative flow, the prototype sentences are then ordered chronologically according to their original positions in the source document. After sorting the indices, the function retrieves the text that corresponds to each of the prototype sentences and joins them into a single, cohesive paragraph.

`keyword_extraction(d,p,I,args)`

The `keyword_extraction` function executes three primary operations: extracting candidate noun phrases via Part-of-Speech (POS) tagging, clustering their vector embeddings to select unique semantic prototypes, and ranking these prototypes by their proximity to the overall document. This yields a concise, ranked, and non-redundant set of keywords summarizing the document's core topics.

The first phase utilizes NLTK to generate a refined list of candidate keywords. The text is tokenized, lowercased, and processed with POS tagging. To capture meaningful concepts, a regular expression parser specifically isolates noun phrases (optional adjectives followed by nouns), inherently filtering out stop words and irrelevant syntax. Finally, candidates exceeding two words are discarded, and the remaining phrases are lemmatized to their base forms and strictly deduplicated.

Once the candidate keywords are isolated, the function employs the same core embedding and clustering architecture utilized in 2.1.1 and 2.1.2. However, there are two notable differences during this phase. First, the algorithm dynamically bounds the target number of clusters (p), which is determined by the `get_optimal_p` function, as detailed in A.1.2. Second, the function simultaneously generates a global document embedding, which is retained to benchmark and rank the prototypes in the final phase.

In the final phase, the algorithm ranks the selected prototypes semantically. It computes the cosine distance between each prototype and the global document vector. By sorting these distances in ascending order, the function prioritizes the terms most aligned with the text's core meaning, returning a ranked, non-redundant set of p keywords.

2.2 Questions

1. Do clustering-guided summarization alters the behavior and efficacy of the IR system? Hypothesize why is this so.

To analyze whether clustering-guided summarization alters the efficacy and behavior of the IR system, we compared the results of our clustering approach against three baselines established in Project I: a lexical model (BM25), our best-performing semantic model (`MiniLM`), and MMR.

To evaluate efficacy, we utilized the `evaluation` function from Project I. The aggregated results across the entire dataset are presented below.

Metric	BM25	`MiniLM`	MMR	Clustering
Macro Precision	0.3803	0.5920	0.3524	0.3399
Macro Recall	0.3183	0.4925	0.3002	0.2639
Macro F_β	0.3435	0.5330	0.3203	0.2982
ROUGE-1	0.5708	0.6369	0.4885	0.4412
ROUGE-2	0.4446	0.5504	0.3453	0.3098
ROUGE-L	0.3975	0.4408	0.3382	0.3169

Table 8. Efficacy comparison between unsupervised (Project I) and unsupervised (Project II) extractive summarization systems.

As observed in Table 8, clustering yields lower raw scores than the `MiniLM` baseline. We hypothesize this drop occurs because clustering shifts the system from a "relevance-maximizer" to a "diversity-enforcer," mirroring MMR. While baselines simply extract the highest concentration of core semantics regardless of repetition, clustering actively penalizes redundancy.

This shift introduces a diversity penalty during evaluation. `MiniLM` scores highly because it continually extracts dense, core information that closely matches reference summaries. By forcing the extraction of a prototype from every cluster, clustering covers more diverse sub-plots but dilutes this core information. Furthermore, boundary constraints amplify this penalty. If the algorithm is forced to generate a high number of clusters, it will inevitably extract unnecessary information.

Finally, the significant drop in ROUGE scores stems from a lexical mismatch. ROUGE strictly evaluates exact word overlap, whereas the clustering algorithm selects centroids based purely on semantic coverage in a vector space, naturally sacrificing the exact lexical matching that standard metrics reward.

Secondly, to further evaluate the system's behavior, we compared the generated summary lengths and the execution times across the entire dataset. The results of this comparison are presented below.

Behavior	BM25	MiniLM	MMR	Clustering
Summary Length	124.9	109.6	96.9	84.2
Execution Time	417.15s	18.21s	125.74s	50.70s

Table 9. Behavior comparison between unsupervised (Project I) and unsupervised (Project II) extractive summarization systems.

As shown in Table 9, the clustering approach produces the most concise summaries by a significant margin. We hypothesize that this is a direct result of how the algorithm dynamically determines the number of sentences to extract. Rather than relying purely on rigid category-based limits or predetermined sentence counts, the clustering approach dictates extraction volume using the Silhouette measure, which is detailed in 2.1.1.

Regarding execution time, clustering optimally balances speed and diversity. By computing the semantic distance matrix only once, it scales better and runs significantly faster than MMR's iterative recalculations. Though naturally slower than the grouping-free `MiniLM` baseline, it vastly outperforms the BM25 lexical approach by bypassing heavy tokenization and term-frequency overhead.

2. How representations and clustering choices impact summarization?

First, we analyze the impact of representation strategies on summarization, followed by the effects of clustering choices. This section explores alternative approaches to those implemented in our pipeline, while the formal justifications for our specific selections are detailed in Sections 2.1.1 and 2.1.2.

Sentence representation fundamentally dictates how the unsupervised system perceives similarity and redundancy. Traditional lexical approaches generate sparse vectors that fail to capture true synonymy, while standard BERT yields highly skewed vector spaces when averaging token embeddings. By employing `Sentence allowbreak Transformer`, we overcome these structural limitations, an improvement empirically validated by Reimers and Gurevych [Reimers and Gurevych 2019].

Furthermore, the specific `SentenceTransformer` model dictates operational efficiency. If a heavier model such as `all-mpnet-base-v2` were utilized, operational efficiency would be severely limited. Empirical comparisons between the two models [SentenceTransformers 2024] demonstrate that while `all-mpnet-base-v2` yields a slightly higher performance score, the `all-MiniLM-L6-v2` model operates over five times faster (14,200 versus 2,800 sentences per second) using only a fraction of memory (80 MB versus 420 MB).

Next, we analyze the impact of clustering choices over five specific parameters: algorithm, distance metric, linkage criterion, dynamic cluster quantity (k), and the final sentence extraction technique.

First, substituting agglomerative clustering with partitional algorithms like k-means introduces instability, as their stochastic initialization yields non-deterministic summaries that fail to map natural semantic contours. Furthermore, relying on a magnitude-sensitive metric like Euclidean distance compromises paraphrase recognition. Because vector magnitude correlates with sentence length, Euclidean metrics falsely separate synonyms, leading to redundant extractions. Lastly, deviating from average linkage to extreme criteria, such as single linkage, severely distorts topic boundaries.

Second, the selection of the default cluster count (k) directly controls the length and fidelity of the produced summaries. Imposing a fixed k severely distorts outputs, padding short texts with redundant and over-condensing lengthy sentences, which omits crucial details. Early iterations of our `sentence_clustering` function focused on dendograms and mitigated this by imposing category-specific distance thresholds. However, these rigid constraints still occasionally ignored a document's unique structure.

Finally, the extraction technique dictates readability. Position-based methods, such as selecting a cluster's first sentence, often capture unresolved transitional hooks rather than complete thoughts, while keyword-density approaches yield convoluted summaries that prioritize vocabulary over narrative flow.

3. Is there redundancy among the extracted keywords? Suggestions for future work?

To explicitly evaluate whether the keyword extraction pipeline produces redundant outputs, a custom function, `analyze_keyword_redundancy`, was implemented to measure both semantic and lexical overlap. Semantic redundancy is measured by encoding keywords into dense vectors using a `SentenceTransformer` and calculating a pairwise cosine Similarity matrix. Lexical redundancy, on the other hand, identifies direct text overlap.

The results from both a single document sample (e.g., document 1) and the entire dataset demonstrate that the `keyword_extraction` function is highly effective in minimizing redundancy. The first document achieved a semantic score of 0.1685 with zero lexical overlaps. This results remained remarkably stable across the entire collection. The corpus yielded a mean semantic score of 0.1769 and a nearly identical median of 0.1763. The tight alignment between the mean and median is a strong indicator of the algorithmic consistency. Furthermore, the lexical redundancy across the corpus was exceptionally low. With only 66 overlapping text pairs identified in total, direct string duplication affected a mere 1.8% of the dataset. This confirms that the POS constraints and deduplication logic robustly isolate unique keywords prior to clustering.

As suggestion for future work, to address the remaining fractional redundancy and further enhance topic diversity, subsequent iterations of the algorithm could incorporate several key refinements. A primary enhancement would be the integration of MMR during the final sorting phase of the `keyword_extraction` function. While the current system ranks prototype keywords purely by their proximity to the global document embedding, MMR would introduce a mathematical penalty for candidates that are too semantically similar to keywords already selected for the final output.

Additionally, the minor 1.8% lexical overlap could be entirely eradicated by implementing a strict substring filter during candidate generation. By explicitly forcing the pipeline to merge or drop nested phrases before the clustering phase begins, the system would guarantee absolute lexical uniqueness across all extracted sets without sacrificing semantic depth.

A Supplementary Results

A.1 Exploratory Data Analysis

Determining the optimal number of clusters

To determine the optimal number of clusters to consider for the `sentence_clustering` function, we rely on the Silhouette Score to dynamically select the ideal grouping for each individual document, following the methodology described in Section 2.1.1. Dendograms are generated to visually validate that the Silhouette approach yields logical results. By examining these side-by-side plots, we can empirically verify if the distinct main branches of the hierarchical tree closely align with the optimal cluster count indicated by the Silhouette curve.

As a demonstrative example, Figure 3 presents the dendrogram and corresponding Silhouette plot for the first document within the Entertainment category.

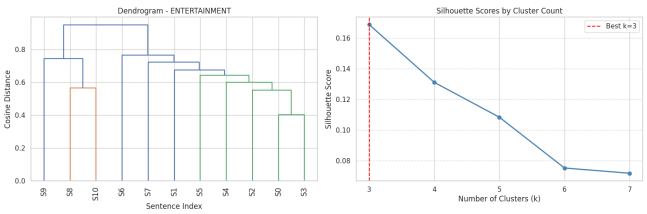


Fig. 3. Dendrogram and Silhouette plot for Document 1 of Entertainment

As demonstrated in Figure 3, the results strongly validate and motivate the use of the dynamic Silhouette approach. The Silhouette curve clearly peaks at an optimal $k = 3$ clusters. When cross-referencing this peak with the structural dendrogram on the left, we can observe that a horizontal threshold near a cosine distance of 0.75 perfectly isolates three distinct main branches. This visual alignment confirms that the Silhouette metric successfully captures the underlying thematic structure of the document, proving its reliability as a default configuration method.

Determining the optimal number p of keywords

Due to the text length variability and domain complexity of our dataset, we decided to investigate the best way to determine the optimal number of keywords, p, that should be extracted. Our goal was to ensure we do not lose any relevant information about a document, while simultaneously avoiding redundant information.

To address this, we implemented a custom function, `get_optimal_p`, to dynamically calculate p for each document. The function evaluates each text based on those two core criteria in the following way: First, the function estimates the document's word count to set an initial baseline for the number of keywords (p): 3 keywords for texts under 100 words, 4 for under 300 words, 6 for under 600 words, and 8 for anything longer. Next, it adjusts for domain complexity by adding 1 to the baseline for the denser topics, while leaving the baseline unchanged for more straightforward topics. Finally, a boundary constraint of $p = \max(2, \min(\text{base_p}, 10))$ is applied to prevent both extreme over-summarization and excessive noise.

To validate this approach, Table 10 and Figure 4 present the calculated keyword targets for a representative document (Document 1) and the entire dataset across each category, respectively.

Doc ID	Category	Word Count	p
bbc_business_001	business	421	6
bbc_entertainment_001	entertainment	190	4
bbc_politics_001	politics	450	7
bbc_sport_001	sport	221	4
bbc_tech_001	tech	672	9

Table 10. Calculated optimal keyword count (p) for the first document of each category, based on word count and domain complexity

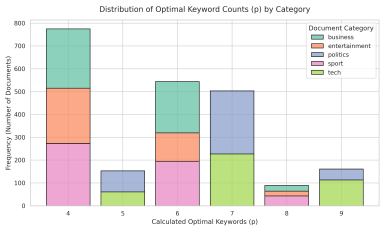


Fig. 4. Calculated optimal keyword count (p) for the overall collection, based on word count and domain complexity

These results provide concrete examples of the two dataset limitations we aim to mitigate. First, regarding text length variability: although *business* is classified as a standard, less complex category, *bbc_business_001* has a surprisingly high word count of 421. Figure 4 confirms this is a trend throughout the dataset rather than an isolated anomaly, demonstrating that even within simpler domains, individual documents can be extensive and require a proportionally higher keyword allowance. Second, the impact of domain complexity is evident when comparing the first documents of the *sport* and *tech* categories. The *sport* article is concise (221 words) and is assigned a p of 4, whereas the *tech* article is highly detailed (672 words), receiving a p of 9. This categorical adjustment is strikingly visible in the global distribution presented by the histogram. The standard categories strictly populate the unadjusted baseline values $p = 4, 6, 8$. In contrast, the denser categories are uniformly shifted to the adjusted $p = 5, 7, 9$. Furthermore, the heavy concentration of *tech* and *politics* documents at the upper end of the spectrum globally validates our hypothesis that complex domains inherently demand a broader extraction target to adequately capture them.

References

Rui Henriques. 2026. Information Retrieval Course Materials. https://fenix.tecnico.ulisboa.pt/disciplinas/RGI610/2025-2026/2-semestre

Hugging Face. 2024. Datasets Documentation: `Dataset.map`. https://huggingface.co/docs/datasets/process#map.

PRI Lab 4. 2026. Lab 4: Aiding IR with Learning Facilities.

Colin Raffel et al. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21 (2020).

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. https://arxiv.org/abs/1908.10084

SentenceTransformers. 2024. Pretrained Models - SentenceTransformers Documentation. https://www.sbert.net/docs/sentence_transformer/pretrained_models.html Accessed: 2026.

Thomas Wolf et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of EMNLP (System Demonstrations)*.